"بسم الله الرحمن الرحيم"

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS5341, Machine Learning and Data Science

Assignment 2

Non-Linear Regression and Logistic Regression

Prepared by:

Khaled Hammad	**ID: 1220857**	**Sec: 1**
Mohammad Shamasneh	**ID: 1220092**	**Sec: 1**

Instructor: Dr. Yazan Abu Farha

Date of submission: 10/23/2025

Table of Contents

Part 1 2
Part 1 – Non-Linear Regression : 2
Part 1_A: ridge regression 3
Part 1_B: RBF 7
Part 2 10
Introduction to logistic Regression 10
Code 11
Result and discussion 15
At linear model 15
At degree 2 16
At degree 5 17
At degree 9 17
Besr model 18
Roc Curve 18
Reference 19

Part 1

Part 1 – Non-Linear Regression :

Theory:

Non-linear regression

Nonlinear regression refers to a broader category of regression models where the relationship between the dependent variable and the independent variables is not assumed to be linear. If the underlying pattern in the data exhibits a curve, whether it's exponential growth, decay, logarithmic, or any other non-linear form, fitting a nonlinear regression model can provide a more accurate representation of the relationship. This is because in linear regression it is pre-assumed that the data is linear [1].

A nonlinear regression model can be expressed as:

$$Y = f(X, \beta) + \epsilon$$

Where,

- $f(X, \beta)$: Regression function
- **X:** This is the vector of independent variables, which are used to predict the dependent variable.
- β : The vector of parameters that the model aims to estimate. These parameters determine the shape and characteristics of the regression function.
- ϵ **:** error term

Types of Non-Linear Regression

There are two main types of Non-Linear regression in Machine Learning:

1. **Parametric non-linear regression** assumes that the relationship between the dependent and independent variables can be modeled using a specific mathematical function.
2. **Non-parametric non-linear regression** does not assume that the relationship between the dependent and independent variables can be modeled using a specific mathematical function [1].

Polynomial Regression

Polynomial regression is a type of nonlinear regression that fits a polynomial function to the data. The general form of a polynomial regression model is:

$$y = \beta_0 + \beta_1 X + \beta_2 X^2 + \ldots\ldots\ldots\ldots + \beta_n X^n$$

where,

- y: dependent variable
- X: independent variable
- $\beta_0, \beta_1, \ldots \beta_n$: parameters of the model
- n: degree of the polynomial [1].

Part 1_A: ridge regression

Regularization

The idea of regularization revolves around modifying the loss function L; in particular, we add a regularization term that penalizes some specified properties of the model parameters

$$L = L(\boldsymbol{w}) + \lambda\, R(\boldsymbol{w})$$

where λ is a scalar that gives the weight (or importance) of the regularization term.

we can choose a regularization term that penalizes the squares of the parameter magnitudes. Then, our regularized loss function is:

$$L_{Ridge}(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n}(\, y_i - \boldsymbol{w}^T \boldsymbol{x}_i\,)^2 + \lambda \sum_{j=1}^{d} w_j^2$$

Note that:

if λ close to zero, we recover the MSE, i.e. ridge regression is just ordinary regression.

If λ is sufficiently large, the MSE term in the regularized loss function will be insignificant and the regularization term will force wridge to be close to zero .

To avoid ad-hoc choices, we should select λ using validation or better cross validation [4].

Ridge regression—also known as L2 regularization—is one of several types of regularization for linear regression models. Regularization is a statistical method to reduce errors caused by overfitting on training data. Ridge regression specifically corrects for multicollinearity in regression analysis. This is useful when developing machine learning models that have a large number of parameters, particularly if those parameters also have high weights [2].

Results:

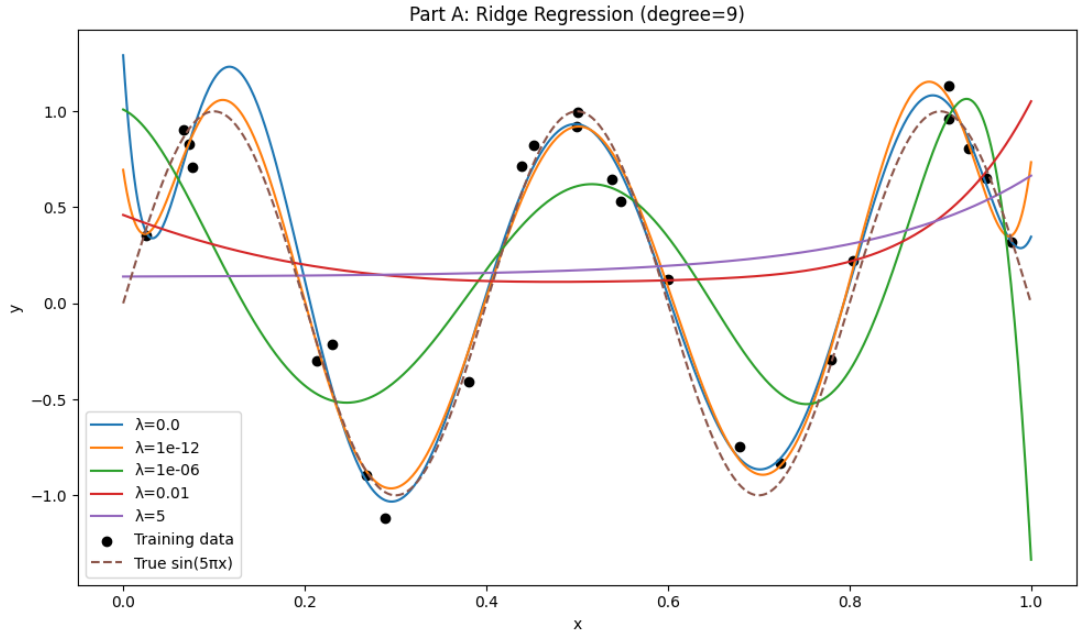

Figure 1:Ridge Regression D9

λ=0

As we see below when we but the value of λ=0 there will be not regularization so we have overfitting.

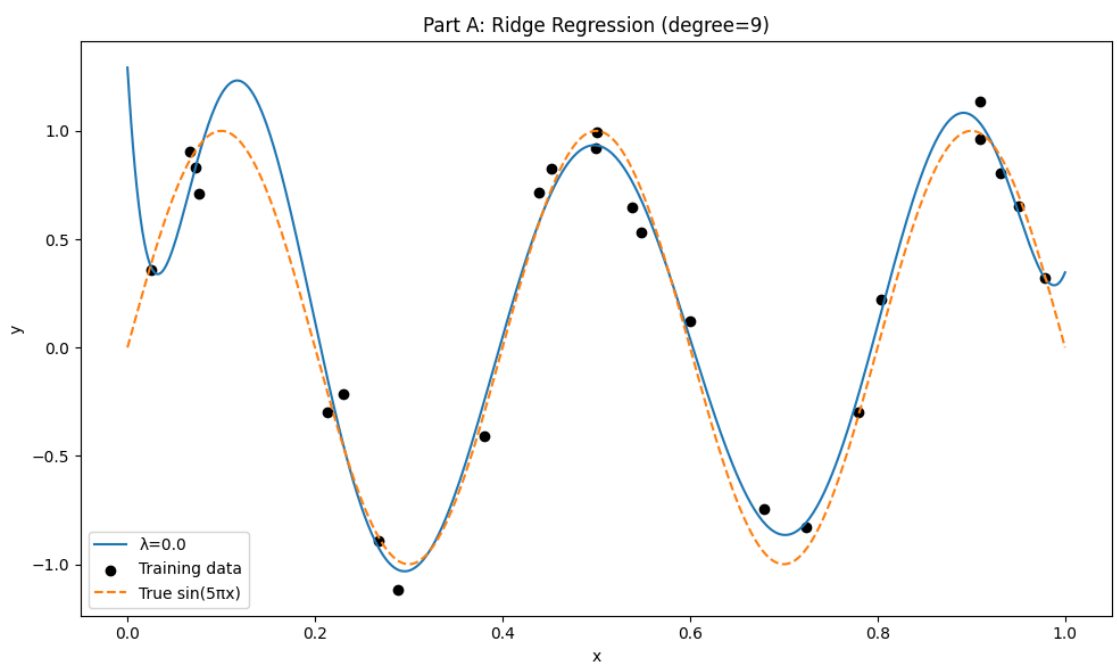

Figure 2:Ridge Regression λ=0

λ=10^{-12}

After many Attempts to find the best value of λ we find that this value is perfect and have a minimum MSE so any value the is over this value will make the model under fitting because as we know increase value of λ increases penalty on large weights so the model will be as linear system of 0 or 1 degree ,and any value that less than this value will make system over fitting because the weight will be large.

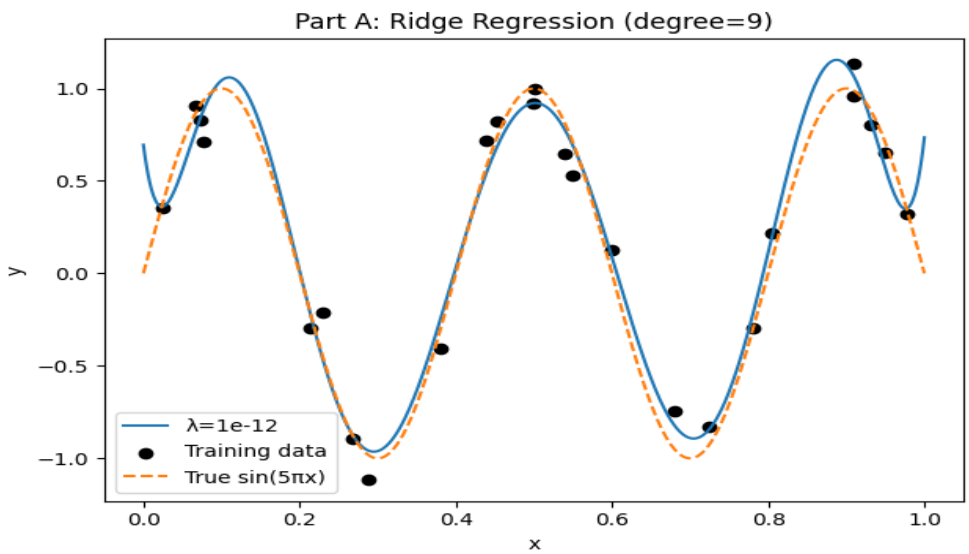

Figure 3:Ridge Regression λ=1e-12.

λ=10^{-6}

(Underfitting)

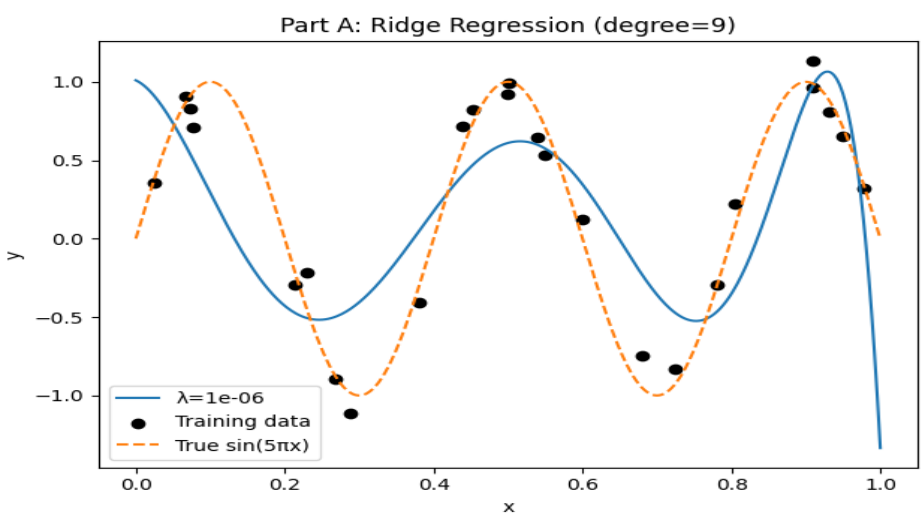

Figure 4:Ridge Regression λ=1e-6.

λ=10^{-2}

(underfitting)

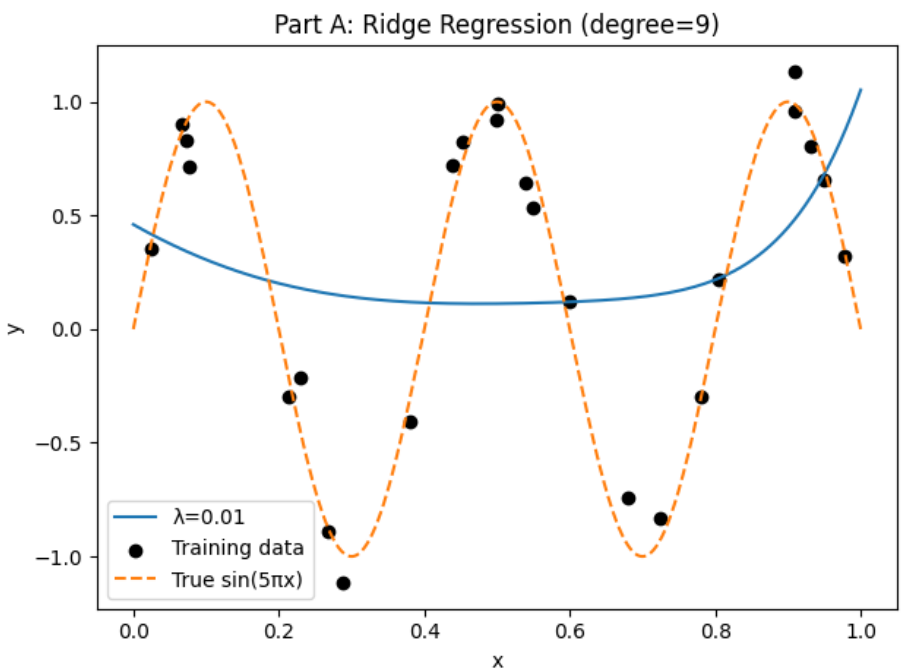

Figure 5:Ridge Regression λ=1e-02.

λ=5

(underfitting) it's shape as polynomial of degree 0.

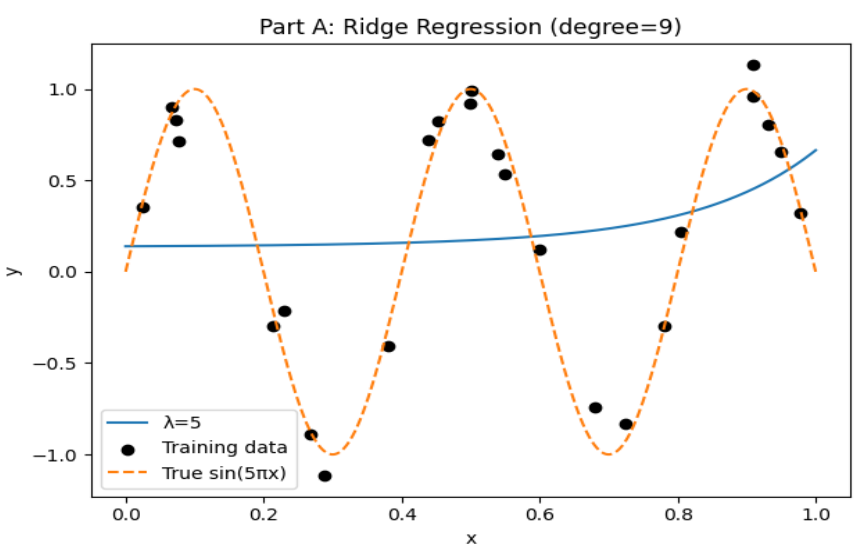

Figure 6:Ridge Regression λ=5.

Part 1_B: RBF

Theory:

A **Radial Basis Function (RBF)** is a type of mathematical function whose value depends only on the distance from a central point. In Machine Learning, RBFs are commonly used to model non-linear relationships, making them effective for tasks like **function approximation**, **classification**, and **regression [3]**.

$$f(x) = e^{\frac{-(x-\alpha)^2}{\lambda}}$$

Where α is the center of each RBF λ controls the width of each RBF.

Why use RBF?

- They avoid problems of high-degree polynomials (exploding values, oscillations).
- They give smooth and stable function approximations.

Result:

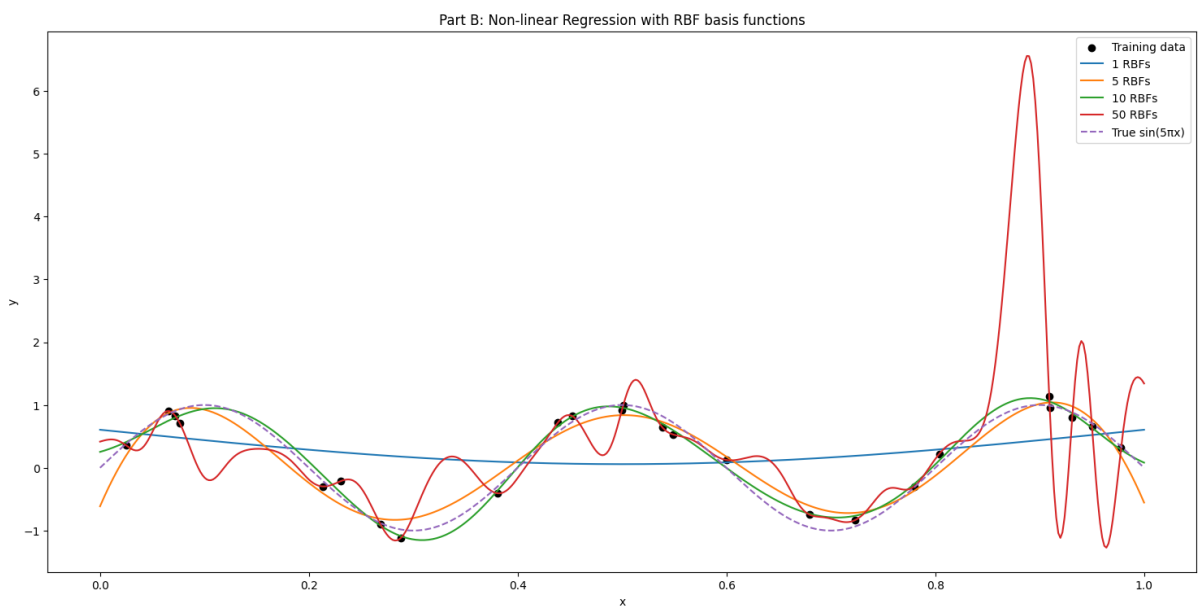

Figure 7: Non-linear Regression with RBF basis functions.

RBF = 1

As we see below when we use RBF=1 the model is being underfitting

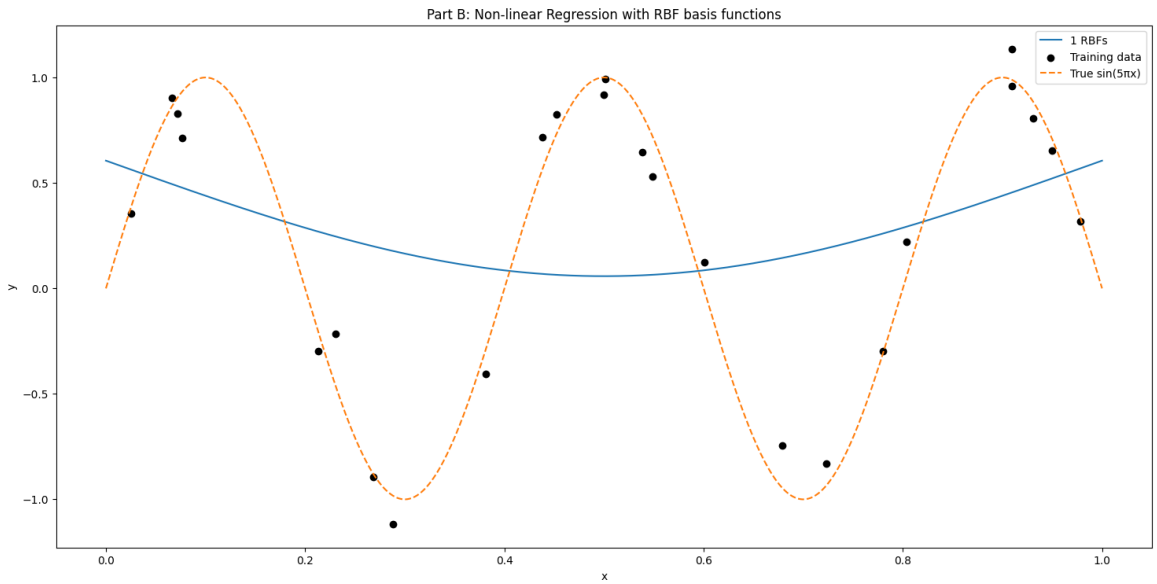

Figure 8:The figure of RPF = 1.

RBF = 5

We observe that model is Improvement after we increase the number of RBF but it still not the perfect.

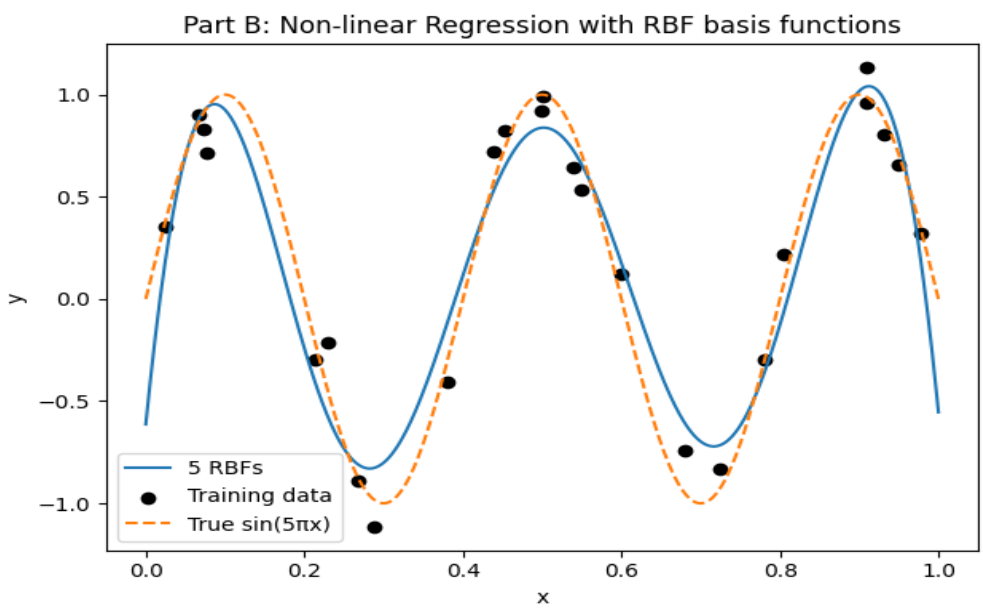

Figure 9:The figure of RPF = 5.

RBF = 10

As we see the model is perfect in training and in validation so this is the best model from these 4 points.

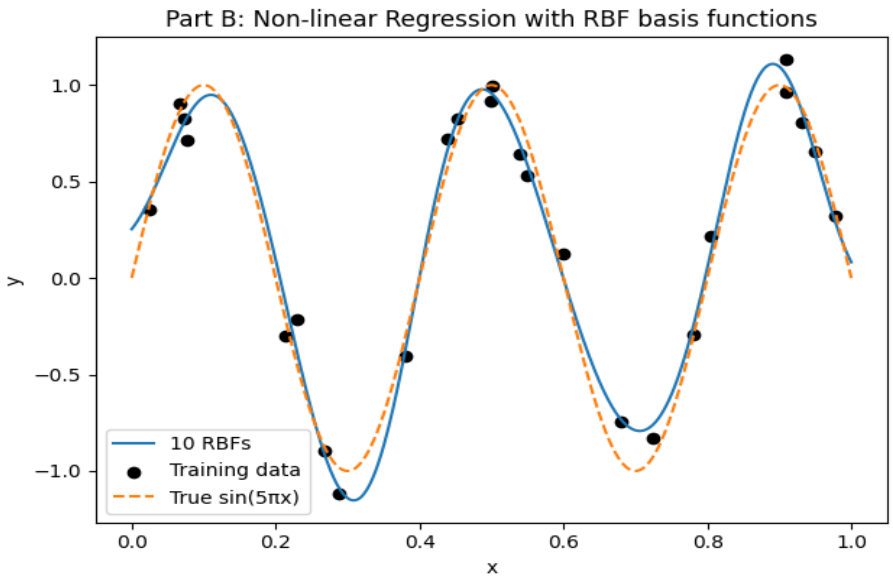

Figure 10:The figure of RPF = 10.

RBF = 50

When we increase the number of RBF to large number, we observe that the model is being good in training data but very bad in validation (Overfitting).

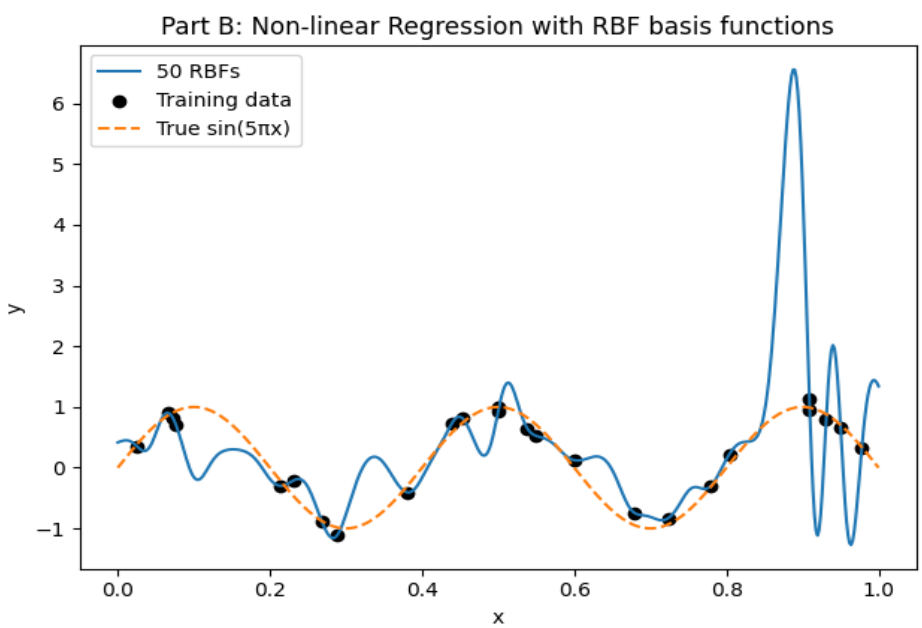

Figure 11:The figure of RPF = 50.

Part 2

Introduction to logistic Regression

Logistic Regression is a supervised machine-learning algorithm used for classification problems, where the dependent (target) variable is categorical rather than continuous. Unlike linear regression which predicts a numerical value, Logistic Regression uses a linear combination of input features and then applies the logistic (sigmoid) function to map that value to a probability between 0 and 1, representing the likelihood that the input belongs to a given class. Based on a threshold often 0.5, the output probability is converted into a class label (e.g. yes/no, 0/1). The model's parameters are typically estimated using statistical techniques (such as maximum likelihood estimation), making Logistic Regression both interpretable and computationally efficient, which is why it remains among the most widely used classification methods across domains such as finance, healthcare, and social sciences.[5]

Logistic regression is inherently a linear classifier: by default it builds a decision boundary as a straight hyperplane in feature space. However, if we transform the original input features for example by adding polynomial features or interaction terms then logistic regression can capture more complex, non-linear relationships between features and the target variable. In that case, although the model inside the transformed feature space remains linear, when you map the decision boundary back to the original feature space it becomes non-linear (a curve, circle, ellipse, or more complex shape depending on the transformations).[6]

In this assignment, we will apply **logistic regression** to predict customer **churn status**. The task involves training and evaluating several logistic regression models, comparing their performance using **accuracy**, **precision**, and **recall**. Specifically, we will work with the following models:

1. **Logistic Regression with a Linear Decision Boundary**: This is the standard logistic regression model.
2. **Logistic Regression with Non-Linear Decision Boundaries**: We will transform the feature space using **polynomial features** of various degrees (2, 5, and 9) to model non-linear relationships.

We will evaluate how these models perform in terms of **accuracy**, **precision**, and **recall** on a test dataset. Additionally, we will explore the impact of **overfitting** and **generalization** on model performance as the polynomial degree increases. Specifically, we'll examine how increasing the degree of polynomial features can lead to models that overfit the training data, potentially reducing their ability to generalize to new, unseen data.

Code

The first part of the code begins by importing several essential Python libraries, where **import pandas** for handle data loading and cleaning, **from sklearn.preprocessing import StandardScaler** for data normalization, **import matplotlib.pyplot as plt** and **from matplotlib.pyplot import title** for plotting, **from sklearn.model_selection import train_test_split** split the dataset into train, validation, and test sets, **from sklearn.preprocessing import PolynomialFeatures** creates polynomial (non-linear) features, **from sklearn.linear_model import LogisticRegression** Imports the Logistic Regression, **from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_curve, auc** Imports evaluation metrics

```
import pandas
from sklearn.preprocessing import StandardScaler
import seaborn as sns
import matplotlib.pyplot as plt
from matplotlib.pyplot import title
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_curve, auc
```

Figure 12: Python libraries

The second part of code we **load** the customer data set, we handling the **missing data** by filling them with the **median**, especially since the data contains outliers that could skew other methods like the mean, For the missing values in the **Income column**, we rely on the **ProductType** column to fill the gaps. We group the data by **ProductType** (which has two categories: **Basic and Premium**) and compute the median Income for each group, then we handle the **outlier** by change its variable not by ignore it I see that this is the best because we suppose that there is an important data in file and if we want to ignore each data, they have outlier we will lose this data that its important.

After we handle missing data and outliers' data we want now to normalized data. We know that Min-Max algorithm is sensitive to outliers but because we cleaned the data from outliers above, we will use it instead of using standardization because it will work as it so they will not be affected.

Note that we normalized the data range between 0-1.

```python
#####################################| read the data |#########################################################

data = pandas.read_csv("customer_data.csv")

#####################################| Handling Missing Data |#################################################

data['Age'] = data['Age'].fillna(data['Age'].median())
data['Tenure'] = data['Tenure'].fillna(data['Tenure'].median())
data['SupportCalls'] = data['SupportCalls'].fillna(data['SupportCalls'].median())
data['Income'] = data.groupby('ProductType')['Income'].transform(lambda x: x.fillna(x.median()))

#####################################| Clean Data from outlier |###############################################

for col in ['Age', 'Tenure', 'SupportCalls','Income']:
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3- Q1
    lowerBound = Q1 - 1.5 * IQR
    upperBound = Q3 + 1.5 * IQR
    medianValue = data[col].median()
    data[col] =data[col].apply(lambda x: medianValue if x < lowerBound or x > upperBound else x) # we change the outlier by median of column instead of remove all row

#####################################| Scaling (Min-Max Normalization) |#######################################

for col in ['Age', 'Tenure', 'SupportCalls','Income']:
    minv = data[col].min()
    maxv = data[col].max()
    data[col] = (data[col]-minv)/(maxv-minv)
```

Figure 13: operation on data

At this part, we creates the input matrix **X** by removing the target and ID, and Creates the target vector **y** (0 = stay, 1 = churn), then we split the data set to: 2500 samples for train, 500 samples for validation, and 500 samples for test.

```python
X = data.drop(["ChurnStatus", "CustomerID" ,], axis=1) # this mean thee data without ChurnStatus is input , axis =1 mean we remove colums / axis =0 mean rows
y = data["ChurnStatus"] # ChurnStatus is the target value

X_train, X_temp, y_train, y_temp = train_test_split(X, y, train_size=2500, random_state=42, shuffle=True)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, train_size=500, random_state=42, shuffle=True)
'''
above we split the data to 2500 training samples, 500 validation samples, and 500 test samples.
'''
```

Figure 14: split data

There we definition function for evaluate model performance by using **accuracy**, **precision**, and **recall** for each **train data** , **validation data** and **test data** .

- **Accuracy** is the percentage of correct predictions (both true positives and true negatives) out of all predictions.
- **Precision** measures how many of the predicted positives were actually correct.
- **Recall** measures how many of the actual positives the model correctly identified.

```python
def evaluate(model, X_train, y_train, X_val, y_val, X_test, y_test):
    results = {}
    y_pred_train = model.predict(X_train)
    y_pred_val = model.predict(X_val)
    y_pred_test = model.predict(X_test)

    results["Accuracy_Train"] = accuracy_score(y_train, y_pred_train)
    results["Accuracy_Val"] = accuracy_score(y_val, y_pred_val)
    results["Accuracy_Test"] = accuracy_score(y_test, y_pred_test)

    results["Precision_Train"] = precision_score(y_train, y_pred_train)
    results["Precision_Val"] = precision_score(y_val, y_pred_val)
    results["Precision_Test"] = precision_score(y_test, y_pred_test)

    results["Recall_Train"] = recall_score(y_train, y_pred_train)
    results["Recall_Val"] = recall_score(y_val, y_pred_val)
    results["Recall_Test"] = recall_score(y_test, y_pred_test)

    return results
```

Figure 15: function for evaluate

Print Function to print the model's **Accuracy**, **Precision**, and **Recall** for the **Train**, **Validation**, and **Test** sets in a clean, organized way.

```python
def print_metrics(results , degree ):
    print(f"\n===== Model (Degree = {degree}) =====")
    metrics_order = ["Accuracy", "Precision", "Recall"]
    sets = ["Train", "Val", "Test"]
    for metric in metrics_order:
        print(f"\n--- {metric} ---")
        for s in sets:
            key = f"{metric}_{s}"
            if key in results:
                print(f"{s}: {results[key]:.4f}")

    print("==========================================\n")
```

Figure 16: print function

This code trains a **linear logistic regression model** and checks how well it performs. It first creates the model and trains it on the training data. Then, using the **evaluate function**, it calculates the model's Accuracy, Precision, and Recall on the train, validation, and test sets. Finally, **print_metrics** displays these results, showing the performance of the degree-1 (linear) model.

```python
model_linear = LogisticRegression(max_iter=500) #create linear LogisticRegression
model_linear.fit(X_train, y_train) # train
metrics_linear = evaluate(model_linear, X_train, y_train, X_val, y_val, X_test, y_test) # evaluate Accuracy (Train / Val / Test) Precision Recall
print_metrics(metrics_linear,1)
```

Figure 17: linear logistic regression

This code trains **logistic regression models with polynomial features** of degrees 2, 5, and 9 to capture non-linear patterns in the data. For each degree:

1. **Polynomial transformation**: **PolynomialFeatures(d)** creates new features up to the specified degree, expanding the feature set for non-linear modeling.
2. **Transform datasets**: The training, validation, and test sets are transformed to include these polynomial features.
3. **Train the model**: A logistic regression model is trained on the transformed training data with a higher **max_iter=2000** to ensure convergence.
4. **Store models and metrics**: Both the trained model and its polynomial transformer are saved in **models_poly**, and performance metrics (**Accuracy**, **Precision**, **Recall**) on training, validation, and test sets are calculated and stored in **metrics_poly**.
5. **Display results**: The **print_metrics** function prints the metrics for each degree, allowing comparison of how increasing polynomial degree affects performance and **overfitting**.

In short: this loop trains and evaluates **non-linear logistic regression models** with increasing polynomial complexity and prints their performance for analysis.

```python
degrees = [2, 5, 9]
metrics_poly = {}
models_poly  = {}

for d in degrees:
    poly = PolynomialFeatures(d)
    X_train_p = poly.fit_transform(X_train)
    X_val_p = poly.transform(X_val)
    X_test_p = poly.transform(X_test)

    model = LogisticRegression(max_iter=2000)
    model.fit(X_train_p, y_train)
    models_poly[d] = (model, poly)
    metrics_poly[d] = evaluate(model, X_train_p, y_train, X_val_p, y_val, X_test_p, y_test)
    ("-----------------------------------------------------------------------------------------------------------")
    print_metrics(metrics_poly[d],d)

    '''
    print(f"Degree {d} : \n {metrics_poly[d]} ")
    print(poly.get_feature_names_out(X_train.columns))
    print(model.coef_)
    print("-----------------------------------------------------------------------------------------------------------")
    print(model.intercept_)
    '''
```

Figure 18: logistic regression models with polynomial

First, the metrics for the **linear model** (degree 1) are added to the **metrics_poly** dictionary. Then, the code identifies the **best model degree** by selecting the model with the highest combined **validation Accuracy and Recall**. Using this **best_degree**, it retrieves the corresponding trained model and polynomial transformer from **models_poly**. Finally, it prints the degree of the best model and its performance metrics, allowing you to see which model best balances performance and generalization.

```python
metrics_poly[1] = metrics_linear

best_degree =  max(metrics_poly,key=lambda d: metrics_poly[d]["Recall_Val"])

best_model, best_poly = models_poly[best_degree]

print("\nBest Model Degree =", best_degree)
print(print_metrics(metrics_poly[best_degree],best_degree))
```

Figure 19: find best model

This code plots the **ROC curve** for the best logistic regression model. First, the test set is transformed using the best model's polynomial features. Then, the model predicts **probabilities** of churn (**predict_proba**), and the **false positive rate (FPR)** and **true positive rate (TPR)** are calculated using **roc_curve**. The **AUC (Area Under the Curve)** is computed to summarize the model's overall performance. Finally, the ROC curve is plotted, showing the trade-off between TPR and FPR, with a diagonal line as a reference for random guessing. The plot title includes the polynomial degree, and the AUC is displayed in the legend to indicate model quality.

```
X_test_best = best_poly.transform(X_test)

y_scores = best_model.predict_proba(X_test_best)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_scores)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2)
plt.plot([0,1], [0,1], "--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title(f"ROC Curve - Logistic Regression (Degree {best_degree})")
plt.legend()
plt.grid()
plt.show()
```

Figure 20: ROC curve code

Note : we cant consider the Accuracy as good measure for this data set, because the most customers do not churn (the data is imbalanced) where 95% will not churn, only 5% churn, so can a model achieve 95% accuracy simply by guessing that everyone will stay.

Out of all the customer who actually left, how many did the model find?

To answer this question we use Recall measure, we need to predict the churn status, because the cost of losing a customer is high. The company prefers to mistakenly expect that the customer will leave and try to satisfy him (low cost), rather than have the customer leave suddenly without the model knowing (Big problem).

Precision measure, when the model said that this customer would leave, was it being truthful? its important if the cost of trying retain the customer is high, but if not high, it's not very important.

Result and discussion

in this experiment, we evaluated the performance of Logistic Regression with a linear decssion boundary models and a non-linear decssion boundary models with varying degrees of polynomial complexity (Degree 2, 5, and 9) to predict customer churn. The goal was to determine how model complexity affects the ability to generalize to unseen data, specifically focusing on the trade-off between bias and variance.

At linear model **:** we see the Accuracy is high for train and validation , and the precision is high, but the recall is 0.43 at train, so there is a some **underfitting** at the model, and its expected, the model is simple and not complex (linear), And unable to grasp the complex relationships that lead to a customer churn, the validation recall is good

$$Z = -0.08259864\, Age * -0.10193146\, Gender * -6.87990142\, Income * -6.96247695\, Tenure * 0.27337461\, ProductType * 0.61098099 * SupportCalls + 1.47403992$$

$$Logistic\ Regression\ EQ = \frac{1}{1 + e^{-Z}}$$

```
===== Linear Model =====

--- Accuracy ---
Train: 0.9740
Val: 0.9840
Test: 0.9760

--- Precision ---
Train: 1.0000
Val: 0.9231
Test: 1.0000

--- Recall ---
Train: 0.4348
Val: 0.6316
Test: 0.4783
==========================================
```

Figure 21: linear model result

At degree 2 **:** as we see very good Accurancy for all set, and high precision, but the recall is Big difference from linear model, at the train we see 0.60 recall, and at validation 0.74 (very good), and this increase between train and validation expresses its no overfitting and no underfitting (generalization), that because its discover nonlinear relationships without great complexity.

```
===== Model (Degree = 2) =====

--- Accuracy ---
Train: 0.9804
Val: 0.9880
Test: 0.9860

--- Precision ---
Train: 0.9459
Val: 0.9333
Test: 1.0000

--- Recall ---
Train: 0.6087
Val: 0.7368
Test: 0.6957
==========================================
```

Figure 22: Degree 2 model result

***At degree 5*:** the accuracy is good, at precision, there is a clear gap (around 6%). The model performs exceptionally well during training, but its performance declines when faced with new data. This is the classic definition of the onset of **overfitting**. The model has begun to "memorize" the training data instead of understanding it, and we have observed a decrease in the Recall metric on the verification data (from 74% in degree 2 to 63% here), the model has become too complex.

```
===== Model (Degree = 5) =====

--- Accuracy ---
Train: 0.9812
Val: 0.9820
Test: 0.9820

--- Precision ---
Train: 0.9146
Val: 0.8571
Test: 0.8889

--- Recall ---
Train: 0.6522
Val: 0.6316
Test: 0.6957
==========================================
```

Figure 23: Degree 5 model result

***At degree 9*:** Accuracy is good, but there is a gap in precision, at train 0.94 and validation 0.85, this significant drop is a red flag. The model had memorized (**overfitting**) the training data and its minute details to such an extent that it began seeing false patterns. When confronted with validation data it had never seen before, it became confused and its precision plummeted, the recall is less than degree 2, this confirms that the excessive complexity of polynomials has led to the preservation of noise rather than the learning of general patterns, making it an unreliable model compared to a second-order model.

```
===== Model (Degree = 9) =====

--- Accuracy ---
Train: 0.9840
Val: 0.9820
Test: 0.9840

--- Precision ---
Train: 0.9412
Val: 0.8571
Test: 0.8947

--- Recall ---
Train: 0.6957
Val: 0.6316
Test: 0.7391
==========================================
```

Figure 24: Degree 9 model result

Besr model :The Logistic Regression model with Degree 2 polynomial features was selected as the optimal model for this experiment, primarily due to achieving the highest Recall score (73.68%) on the Validation set. Since the critical objective in customer churn prediction is to identify the maximum number of at-risk customers, this superior Recall makes it the most practically valuable model. Technically, this model successfully navigated the bias-variance tradeoff; it outperformed the linear model (which suffered from underfitting) while avoiding the overfitting and instability observed in higher degrees (5 and 9). This demonstrates that quadratic and interaction terms were sufficient to capture the non-linear patterns in the data without introducing excessive complexity.

```
Best Model Degree = 2

===== Model (Degree = 2) =====

--- Accuracy ---
Train: 0.9804
Val: 0.9880
Test: 0.9860

--- Precision ---
Train: 0.9459
Val: 0.9333
Test: 1.0000

--- Recall ---
Train: 0.6087
Val: 0.7368
Test: 0.6957
========================================
```

Figure 25: best model

Roc Curve

This figure illustrates the ROC Curve for the selected **Degree 2** model. The curve's steep ascent towards the top-left corner indicates exceptional classification performance. An **AUC of 0.982** implies that the model has a **98.2% probability** of correctly distinguishing between a churning customer and a non-churning one. This confirms that the model achieves a high True Positive Rate (Recall) while maintaining a minimal False Positive Rate, validating its robustness and suitability for the task.

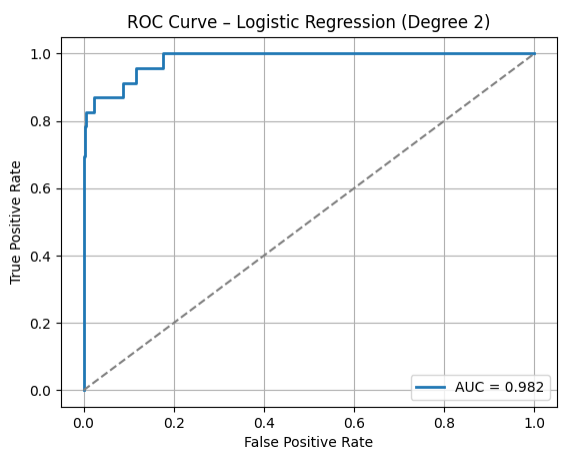

Figure 26: ROC curve

Reference

[1]: Understanding Nonlinear Regression with Examples - GeeksforGeeks

[2]: What Is Ridge Regression? | IBM

[3]: Radial Basis Function in Machine Learning

[4]: Slide(Model Selection and Regularization)

[5]: BMI, What is logistic regression, Click Here

[6]: geeksforgeeks, logistic regression with polynomial feature, Click Here